

WEEK 2 – PRODUCT AND CATEGORY MANAGEMENT SYSTEM

Scenario:

The e-commerce organization needs to store and manage details about its products and categories. Every product is linked to a particular category, while the system keeps track of important information such as product name, price, and available stock.

1. Design Product and Category Tables

Creating Category Table:

Category table was created with the attributes Category_ID, Category_Name and Description. Category_ID is used to uniquely identify each category.

```
SQL> CREATE TABLE CATEGORY(Category_ID VARCHAR(7),Category_Name VARCHAR(20),Description VARCHAR(50));  
Table created.
```

Creating Product Table:

A Product table was created with the attributes Product_ID, Product_Name, Category_ID, Price and Stock_Quantity. This table stores the details of each product.

```
SQL> CREATE TABLE PRODUCT(Product_ID VARCHAR(7),Product_Name VARCHAR(20),category_ID VARCHAR(7),Price NUMBER(9,2),Stock_quantity NUMBER);  
Table created.
```

2. Define Relationships and Constraints

Defining Primary Key:

Primary keys were added to both tables. Category_ID acts as the primary key in the Category table, while Product_ID acts as the primary key in the Product table. A primary key uniquely identifies each record.

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);  
Table altered.
```

Creating Foreign Key Relationship:

The relationship between Category and Product is One-to-Many (1:N). One category can contain many products. The Category_ID in the Product table acts as a foreign key and connects it with the Category table.

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(category_ID);  
Table altered.
```

Adding NOT NULL Constraint:

The NOT NULL constraint was applied to the Category_Name column. It ensures that the category name cannot be left empty.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

Adding CHECK Constraint:

A CHECK constraint was added to ensure that the product price is greater than zero.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (price > 0);  
Table altered.
```

Another CHECK constraint was added to ensure that the stock quantity is not negative.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

Adding UNIQUE Constraint:

A UNIQUE constraint was applied to the Category_Name column. It prevents duplicate category names from being entered into the table.

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);  
Table altered.
```

3. Inserting Records

Sample records were inserted into the Category table. The categories include Travel, Sports and Automotive, along with their respective descriptions.

```

SQL> INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description');
Enter value for category_id: 4053
Enter value for category_name: Travel
Enter value for description: Travel and tourism product
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('4053', 'Travel', 'Travel and tourism product')

1 row created.

SQL> /
Enter value for category_id: 4054
Enter value for category_name: Sports
Enter value for description: Sports and fitness products
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('4054', 'Sports', 'Sports and fitness products')

1 row created.

SQL> /
Enter value for category_id: 4055
Enter value for category_name: Automotive
Enter value for description: Car and bike accessories
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('4055', 'Automotive', 'Car and bike accessories')

1 row created.

```

```

SQL> /
Enter value for category_id: 4056
Enter value for category_name: pet
Enter value for description: Food and toys accessories
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('4056', 'pet', 'Food and toys accessories')

1 row created.

SQL> /
Enter value for category_id: 4057
Enter value for category_name: Gardening
Enter value for description: Gardening tools and supplies
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('4057', 'Gardening', 'Gardening tools and supplies')

1 row created.

```

PRODUCT TABLE:

```

SQL> INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity);
Enter value for product_id: 501
Enter value for product_name: Travel Backpack
Enter value for category_id: 4053
Enter value for price: 1800
Enter value for stock_quantity: 25
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('501', 'Travel Backpack', '4053', 1800, 25)

1 row created.

SQL> /
Enter value for product_id: 502
Enter value for product_name: Football
Enter value for category_id: 4054
Enter value for price: 1200
Enter value for stock_quantity: 30
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('502', 'Football', '4054', 1200, 30)

1 row created.

SQL> /
Enter value for product_id: 503
Enter value for product_name: car cover
Enter value for category_id: 4055
Enter value for price: 2500
Enter value for stock_quantity: 20
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('503', 'car cover', '4055', 2500, 20)

1 row created.

```

```

SQL> /
Enter value for product_id: 504
Enter value for product_name: Pet food
Enter value for category_id: 4056
Enter value for price: 900
Enter value for stock_quantity: 40
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('504', 'Pet food', '4056', 900, 40)

1 row created.

```

After inserting the records, the contents of both tables can be displayed to verify that the data has been stored successfully.

```
SQL> SELECT * FROM Category;
```

```

CATEGOR
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
4053
Travel
Travel and tourism product

4054
Sports
Sports and fitness products

CATEGOR
-----
CATEGORY_NAME
-----
DESCRIPTION
-----

4055
Automotive
Car and bike accessories

```

```

4056
pet

CATEGOR
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
Food and toys accessories

4057
Gardening
Gardening tools and supplies

```

```
SQL> SELECT * FROM Product;
```

PRODUCT	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
501	Travel Backpack	4053	1800	25
502	Football	4054	1200	30
503	car cover	4055	2500	20
504	Pet food	4056	900	40

4. CRUD Operations:

CRUD stands for **Create, Read, Update, and Delete**. These are the four fundamental actions used to handle and maintain records in a database. They help users manage product details effectively from adding new information to removing existing records.

Insert Record:

The Insert operation is used to enter a new product into the Product table. It helps in storing the details of products that are newly added or available.

```
SQL> INSERT INTO Product (Product_ID,Product_Name,Category_ID,Price,Stock_Quantity) VALUES ('505','Pet food','4056',900,40);  
1 row created.
```

Update Price: The Update operation is used to modify existing product information. In this case, it can be used to change the price of a product when its selling price is revised.

```
SQL> UPDATE Product SET price = 50000 WHERE Product_Name = 'car cover';  
1 row updated.
```

Delete a Product: The Delete operation is used to remove a product entry from the Product table. It is useful when a product is no longer sold, available, or required in the database.

```
SQL> DELETE FROM Product WHERE Product_Name = 'Football';  
1 row deleted.
```

5. Generate Category-wise Product Reports:

- **Count number of products in each category:** This query is used to find the total number of products available in each category. It helps in understanding how many products are listed under different categories.

```
SQL> SELECT c.Category_Name,  
2      COUNT(p.Product_ID) AS Product_Count  
3 FROM Product p  
4 JOIN Category c  
5 ON p.Category_ID = c.Category_ID  
6 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
PRODUCT_COUNT
```

```
Automotive
```

```
1
```

```
Travel and tourism product
```

```
1
```

```
pet
```

```
2
```

- **Find highest-priced product in each category:** This query is used to identify the most expensive product available in every category. It helps to compare the highest-priced products among the different categories.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_Price
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 WHERE p.Price = (SELECT MAX(p2.Price)
5 FROM Product p2
6 WHERE p2.Category_ID = p.Category_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Travel and toursim product	Travel Backpack	1800
Automotive car cover		50000
pet	Pet food	900

- **Display categories having more than 5 products:** This query is used to show only the categories that contain more than five products. It helps identify categories with a higher number of products.

```
SQL> SELECT c.Category_Name
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 GROUP BY c.Category_Name
5 HAVING COUNT(p.Product_ID) > 2;
```

no rows selected

- **Find average product price category-wise:** This query is used to calculate the average price of products in each category. It helps understand the typical price of products across different categories.

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVG_PRICE
Automotive	50000
Travel and toursim product	1800
pet	900

Conclusion:

The Product and Category Management System offers an organized method for managing e-commerce product details. The use of relationships and constraints helps maintain accurate and consistent data. CRUD operations allow users to add, view, modify, and remove product records. Category-based SQL queries also help in obtaining useful details about products and their pricing.